

STEM · TEM · ELECTRON MICROSCOPY

AI Agents for Electron Microscopy

Architecture · Frameworks · Prompting · Fine-Tuning

— Sheryl L. Sanchez, PhD

—
UNIVERSITY OF
TENNESSEE

Materials Science &
Engineering

What We'll Cover

A structured journey through AI agent design for electron microscopy

01 What Is an AI Agent?

Definitions, loops, ReAct

03 Frameworks & smol-agents

smol-agents, LangGraph, MCP

05 Prompt Engineering

System prompts, CoT, structured output & more

07 Building Your Agent

RAG, pipelines, run it end-to-end

02 Why Agents for Microscopy?

Challenges & capabilities

04 Agent Architecture

Layers, memory, tools, runnable code

06 Fine-Tuning with LoRA

Adapters, QLoRA, checkpoints, updates

Python examples throughout

smol-agents · LangGraph · chromadb · peft

01

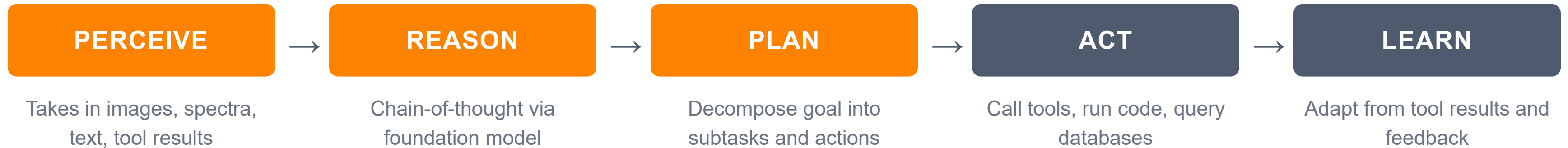
What Is an AI Agent?

Foundations, definitions, and the ReAct loop

01

Defining an AI Agent

An agent perceives its environment, reasons about what to do next, and acts autonomously toward a goal



Why a loop, not a single call?

A Single LLM Call

- One forward pass: image or question in, text out
- No tools — can't measure d-spacing or query a database
- No memory of prior tool results within the task
- Can't verify or correct its own answer

Best for: a single caption, a quick classification, a one-shot summary

An AI Agent

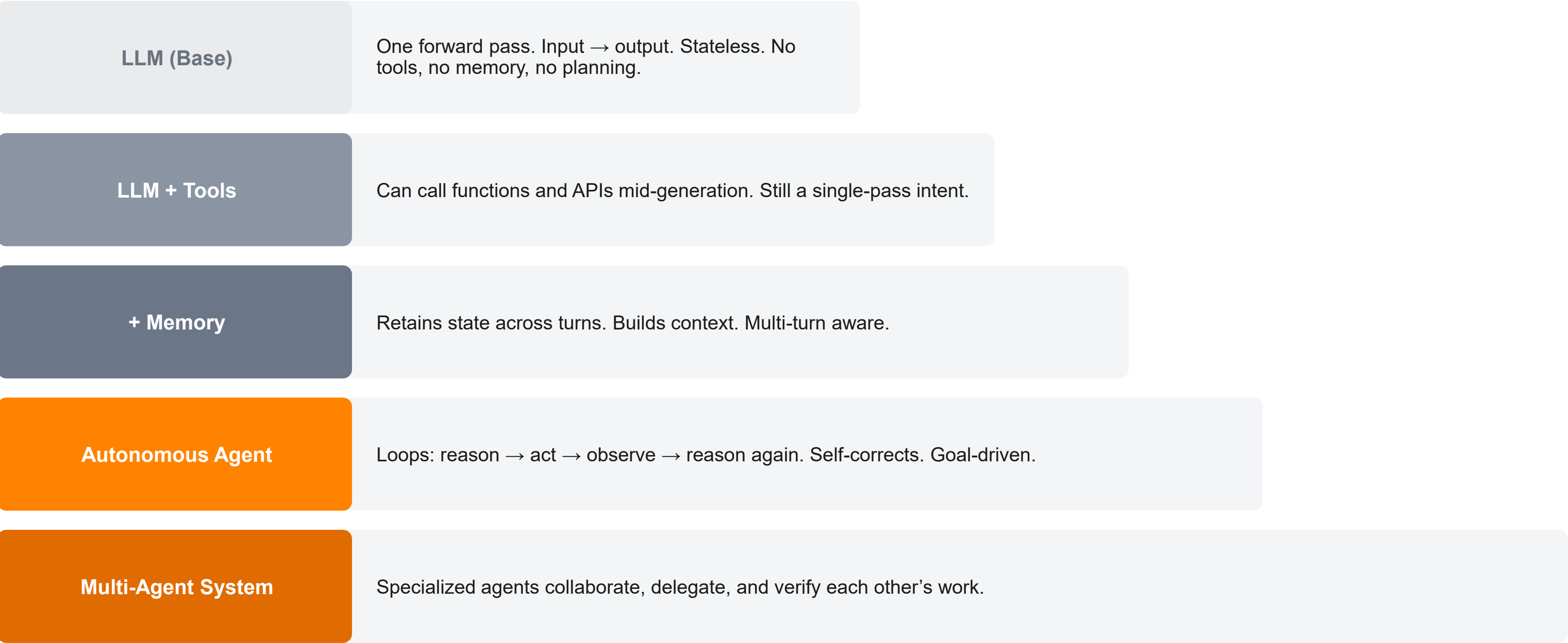
- Loops: reason → act → observe → repeat (see next slide)
- Calls tools: measures lattice spacing, runs diffraction analysis, reads files
- Carries state across many steps in one session
- Self-corrects using real tool feedback, not just its own guess

Best for: full microscopy analyses, literature-grounded reports

Key distinction: Agents **decide what to do next** and loop until the goal is complete — they are not just one-shot predictors.

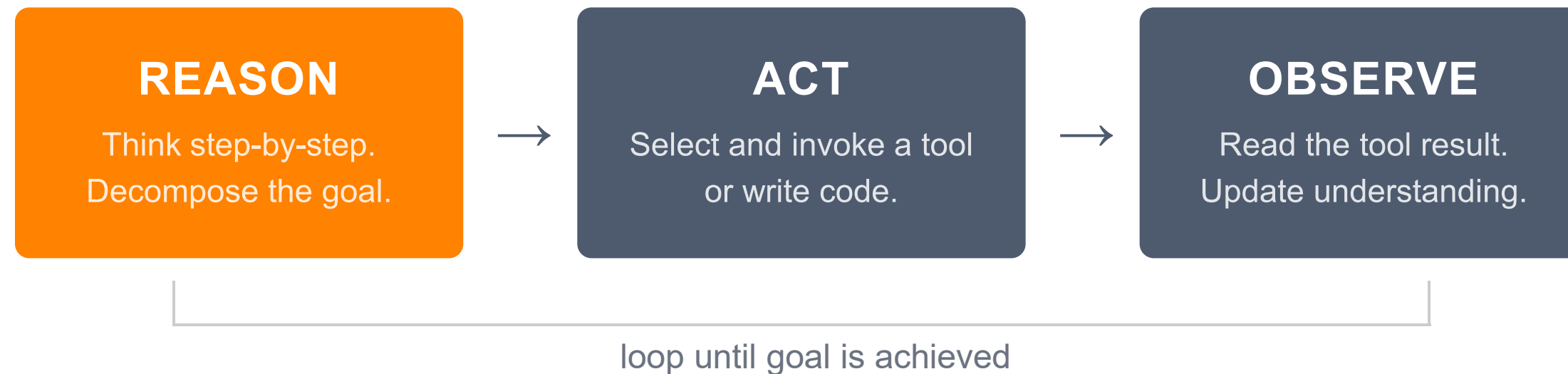
From LLM to Agent: The Key Leap

Each layer adds one capability — the jump to a loop is what makes it an agent



The ReAct Loop

The fundamental execution cycle for all agentic behavior — Reason → Act → Observe



Key Properties

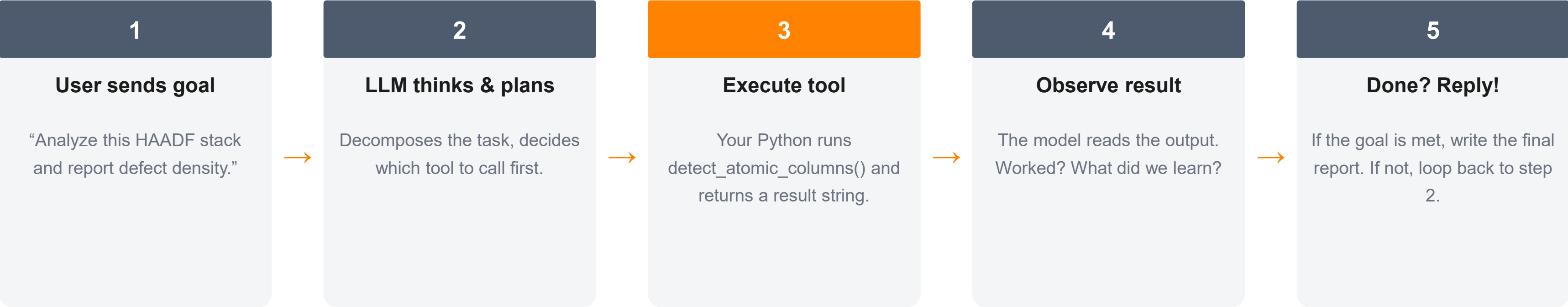
- Self-correcting on tool failure
- Stops when goal is achieved
- Each loop = one LLM call
- Trace is fully auditable
- Max steps prevents runaway

A Worked Example

```
REASON    "Hexagonal lattice, bright Ga columns. Need spacing to confirm orientation."  
ACT       detect_atomic_columns("GaN_HAADF.tif")  
OBSERVE   247 columns found, spacing = 0.195 nm  
REASON    "0.195 nm matches GaN [0001]. Checking for defects next..."  
# loop continues: ACT -> detect_defects(...) -> OBSERVE -> ... until done
```

The Agent Loop, Step by Step

Five stages of a single run — the model plans, your code acts, the loop repeats



← steps 2–4 repeat until the goal is complete (or a max-steps limit is hit)

Key insight: the model never executes code directly. It outputs a JSON tool-call; your Python reads it, runs the function, and feeds the result back. That boundary keeps you in control of the instrument.

02

Why Agents for Electron Microscopy?

The data challenge and what agents enable

02

100s

GB per TEM session

100K+

EM papers published

Hours

to manually analyze one
dataset

The STEM/TEM Data Challenge

Data Volume

Modern microscopes generate gigabytes of images, spectra, and diffraction patterns every session. Manual review creates analysis backlogs.

Expert Bottleneck

Interpretation of HAADF, EELS, EDX, and diffraction data requires years of specialized training. A single expert can only do so much.

Multi-Modal Complexity

Images + EELS + EDX + diffraction patterns must be correlated. No single expert holds all the cross-domain knowledge needed.

What Agents Enable for Microscopy

From raw data acquisition to interpreted, reproducible results

Automated Image Analysis

Segment atomic columns, measure d-spacings, identify defects and dislocations — without manual clicking

Spectral Interpretation

Identify EELS edges, quantify elemental composition from EDX maps, match to reference databases automatically

Literature-Grounded Context

Retrieve relevant papers via RAG to contextualize observations — agents cite evidence, not just parameters

Hypothesis Generation

From observed defect structures, suggest next experiments, imaging conditions, or sample preparation steps

Multi-Instrument Orchestration

Coordinate STEM, EELS, EDX, and simulation tools (VESTA/QSTEM) in a single automated workflow

Reproducible Analysis Pipelines

Full audit trails — every tool call, parameter, and intermediate result logged. Re-runnable from any checkpoint.

03

Frameworks & smol-agents

Where to get agents and how to get started quickly

03

The Agent Ecosystem

Don't build from scratch — choose a framework that fits your workflow

Framework	Style	Key Strength	Best For Science
smolagents (HF)	Code generation	Lightweight, local models, CodeAgent	Quick prototyping, local/secure data
LangGraph	State machine	Fine-grained control, branching logic	Complex multi-step analysis workflows
AutoGen	Multi-agent conversation	Built-in debate and verification	Peer-review style agent verification
CrewAI	Role-based teams	Intuitive roles, task delegation	Structured research pipelines
PydanticAI	Type-safe Python	Type-safe outputs, dependency injection	Strict schema-checked analysis reports
Haystack Agents	Pipeline + RAG-native	Built-in retrievers, rankers, document stores	Literature-heavy retrieval pipelines
OpenAI Agents SDK	Lightweight handoffs	Simple agent handoff and guardrail primitives	Small teams standardizing on one vendor

For local, on-premise science with sensitive microscopy data: **smolagents + Qwen2.5-VL** or **LangGraph + Llama 3.1 70B** are strong starting points.

smol-agents: Lightweight Agent Building

HuggingFace's minimal, open-source agent library — `pip install smolagents`

Why smol-agents for science?

Runs Locally

Works with HuggingFace models on your own GPU cluster — no sensitive microscopy data leaves your lab

Minimal Dependencies

Tiny footprint — easy to deploy on HPC clusters or lab workstations with existing Python environments

Scientific Stack Friendly

CodeAgent can call numpy, scipy, scikit-image, hyperspy, ncempy directly — no special wrappers needed

Two agent types

CodeAgent

Generates and executes Python code as actions. The agent writes `analyze_stem_image("file.tif")` and runs it. More flexible for scientific computing.

ToolCallingAgent

Uses JSON function calling (like standard API tool use). More predictable, easier to audit. Good for well-defined instrument interfaces.

Both types support **@tool** decorator, **HfApiModel**, **TransformersModel**, and multi-agent via **ManagedAgent**

Getting Started in 3 Commands

1. Install

```
pip install smolagents[transformers]
```

2. Pick a model

```
HfApiModel(  
    "Qwen/Qwen2.5-VL-7B-Instruct")
```

3. Run it

```
agent.run(  
    "Analyze GaN_HAADF.tif")
```


smol-agents: CodeAgent in Practice

Define a tool with a docstring — the agent reads it and writes code to call it

@tool decorator

Any Python function becomes an agent tool. The docstring IS the tool description — write it carefully.

HfApiModel

Connects to any HuggingFace model. Use Qwen2.5-VL for vision tasks (it can see your microscopy images).

agent.run()

Pass a plain-language task. The CodeAgent reasons, writes Python, executes it, observes results, and loops until done.

```
from smolagents import CodeAgent, HfApiModel, tool
import json
@tool
def analyze_stem_image(path: str, min_dist: int = 5) -> str:
    """Detect atomic columns in a STEM/HAADF image.
    Args: path: path to .tif or .dm4 file
    Returns: JSON with column count and positions
    """
    from skimage import io, feature
    img = io.imread(path, as_gray=True)
    peaks = feature.peak_local_max(img, min_distance=min_dist)

    return json.dumps({"count": len(peaks), "peaks" : peaks.
        tolist()})
model = HfApiModel("Qwen/Qwen2.5-VL-7B-Instruct")
agent = CodeAgent(tools=[analyze_stem_image], model=model)
result = agent.run(
    "Analyze GaN_HAADF.tif - calculate atomic column density"
)
```

LangGraph: State-Machine Agents

When the analysis has branches, retries, or needs to pause for a human, model it as an explicit graph

StateGraph

Nodes are plain functions; edges (including conditional edges) decide where control flows next. The graph IS the agent loop, made explicit.

Checkpointter

Saves the full state after every node. A multi-hour microscopy session can crash, pause for a human, and resume exactly where it left off.

```
from langgraph.graph import StateGraph, END
from langgraph.checkpoint.memory import MemorySaver

llm = ChatAnthropic(model=MODEL).bind_tools(em_tools)

def call_model(state):
    response = llm.invoke(state["messages"])
    return {"messages": [response]}

def should_continue(state):
    last = state["messages"][-1]
    return "tools" if last.tool_calls else END

graph = StateGraph(dict)
graph.add_node("agent", call_model)
graph.add_node("tools", tool_node)
graph.add_conditional_edges("agent", should_continue)
graph.add_edge("tools", "agent")

app = graph.compile(checkpointer=MemorySaver())

# resume: app.invoke(..., config={"configurable": {"thread_id": "run_42"}})
```

MCP Servers: A Standard for Tool Access

Instead of wiring every tool into every agent by hand, plug into a shared, growing ecosystem

What MCP Is

An open protocol (from Anthropic) that standardizes how an agent connects to tools, data, and prompts — one interface instead of custom code per integration.

Client-Server Architecture

Your agent (the client) connects to one or more MCP servers. Each server exposes its tools over the same standard interface, regardless of who built it.

Tools, Resources, Prompts

A server can expose three things: tools (callable functions), resources (readable data like instrument logs or files), and prompts (reusable templates).

Write Once, Plug In Anywhere

An MCP server wrapping your microscope control software works whether your agent is built with smol-agents, LangGraph, or called directly — no rewrites.

STEM/TEM Examples

Wrap instrument control, a literature search API, or HPC job submission as an MCP server — every agent in the lab inherits that capability instantly.

Scoped Access & Auditing

Each server runs as its own process with limited permissions. You decide exactly which tools and data each agent can reach, and every call is logged.

Project Structure for a Microscopy Agent

A clean, lab-ready layout — separate prompts, tools, and memory from the agent loop

prompts/ & tools/

System prompts and tool docstrings live outside Python — edit instrument protocols without redeploying code.

memory/ & .env

Session history persists across runs. API keys and instrument credentials never get committed to git.

```
em_agent/  
|  
├─ prompts/  
|   └─ system.txt  
|   └─ examples/  
|  
├─ tools/  
|   └─ __init__.py  
|   └─ load_em_data.py  
|   └─ analyze_eels.py  
|  
├─ memory/  
|   └─ history.py  
|  
├─ agent.py  
├─ config.py  
├─ main.py  
└─ .env  # never commit!
```

Loading Prompts in Code

A tiny config module reads your prompt files and central settings — import it everywhere

PROMPTS_DIR / name

`load_prompt("system.txt")` reads the text file at runtime, so editing the prompt never touches Python.

Secrets via .env

`load_dotenv()` pulls your API key from a `.env` file. The key stays out of code and out of version control.

```
# config.py

from pathlib import Path
import os
from dotenv import load_dotenv

load_dotenv()

PROMPTS_DIR = Path("prompts")
MODEL        = "claude-opus-4-20250514"
API_KEY      = os.getenv("ANTHROPIC_API_KEY")

def load_prompt(name: str) -> str:
    """Load a prompt file by name."""
    path = PROMPTS_DIR / name
    return path.read_text(encoding="utf-8").strip()

# usage in agent.py:
system = load_prompt("system.txt")

# inject values: prompt.format(sample_id=sid)
```

04

Agent Architecture

The layers, memory systems, and tools every agent needs

04

The Five Architecture Layers

Five interlocking layers — bidirectional information flow between all of them



Memory Systems

Agents need different memory types to handle long-horizon microscopy workflows

In-Context Memory

Working Memory

Lives in the LLM's context window

Holds current image, recent tool results, working hypotheses

Fast — zero latency access

Limited to 8K–200K tokens; lost when session ends

External Memory

Long-Term / Semantic

Vector database (FAISS, ChromaDB, Weaviate)

Stores embeddings of EM papers, crystal protocols, reference spectra

Retrieved via similarity search (RAG)

Persists across sessions; cite retrieved sources

Episodic Memory

Experience Store

Records past agent runs and their outcomes

Structured log: tool call → result → quality score

Agent learns what worked for which sample type

Can be distilled into fine-tuning data over time

Saving Session History

The model has no memory of its own — you persist the message history so sessions survive a restart

One record per session

Each microscopy session is keyed by a `session_id`. Saving after every turn means a crash loses nothing.

JSON now, database later

A JSON file is perfect for learning. For many concurrent sessions, swap in SQLite or Redis — same interface.

```
# memory/history.py

import json
from pathlib import Path

STORE = Path("memory/store.json")

def load_history(session_id: str) -> list:
    if not STORE.exists(): return []
    data = json.loads(STORE.read_text())
    return data.get(session_id, [])

def save_history(session_id, history):
    data = json.loads(STORE.read_text()) \
        if STORE.exists() else {}
    data[session_id] = history
    STORE.write_text(
        json.dumps(data, indent=2))

# call save_history() after every loop turn
```

Tool Design for STEM/TEM

Each tool = a Python function + a precise docstring. Provide 5–15 well-documented tools per agent.

`load_em_data(path)`

Load and preprocess .dm4, .tif, .ser, .emi files;
return normalized numpy array + metadata

`detect_atomic_columns(img)`

Template-matching or ML-based peak detection;
return column positions and Z-contrast values

`measure_lattice(positions)`

Compute d-spacings, bond angles, and unit cell
parameters from detected column positions

`analyze_eels(spectrum)`

Identify ionization edges, fit background, quantify
elemental composition from EELS spectrum data

`query_structure_db(formula)`

Search a reference structure database for known
phases matching composition and symmetry

`search_em_literature(query)`

RAG retrieval from indexed EM papers; return top-
k relevant passages with DOIs for citation

A Complete Tool: detect_atomic_columns.py

Every tool file holds two things — the SCHEMA the model reads and the run() your code executes

SCHEMA = declaration

Name, plain-English description, and typed inputs. The model reads this to decide when and how to call the tool.

run() = execution

Your Python does the real work and always returns a string — including on error, so the agent can react and retry.

```
# tools/detect_atomic_columns.py

import numpy as np

SCHEMA = {

    "name": "detect_atomic_columns",
    "description": "Find atomic column
        positions in a HAADF-STEM image.",
    "input_schema": {"type": "object",
        "properties": {"path": {"type":
            "string"}}, "required": ["path"]}}

def run(path: str) -> str:
    """Return column count and spacing."""
    try:
        img = load_image(path)
        cols = find_peaks(img)
        d = mean_spacing(cols)
        return f"{len(cols)} cols, d={d:.3f} nm"
    except Exception as e:
        return f"Error: {e}"
```


Registering Your Tools

Every EM tool has two parts: a schema the model reads, a handler your code runs

JSON Schema

Sent to the LLM so it knows the tool exists — name, description, parameter types, required fields.

Python Handler

The function your code runs when called. Takes the model's inputs, returns a string result (or error).

```
from . import load_em_data, detect_columns, analyze_eels

# Schemas -> passed to the API as tools=
ALL_SCHEMAS = [
    load_em_data.SCHEMA,
    detect_columns.SCHEMA,
    analyze_eels.SCHEMA,
]

# Handlers -> map tool name to function
HANDLERS = {
    "load_em_data": load_em_data.run,
    "detect_columns": detect_columns.run,
    "analyze_eels": analyze_eels.run,
}

# New tool in 3 steps:
# create the .py file, import it, add to both dicts
```

05

Prompt Engineering

System prompts, chain-of-thought, and few-shot examples

05

System Prompts & Instruction Design

The system prompt is the agent's permanent instruction set — write it like a job description for a scientist

Establish Role & Expertise

Tell the model who it is, its domain, and its institutional context. This grounds all reasoning.

Define the Analysis Protocol

Enumerate the steps to follow: describe → measure → retrieve → interpret → cite. Makes behavior predictable.

Set Uncertainty Rules

Require confidence ratings. Forbid guessing element identities without tool verification. Mandate citations.

Invoke Chain-of-Thought

"Think step by step before answering." Dramatically improves accuracy on multi-step reasoning tasks.

```
# system.txt — STEM/TEM analysis agent
```

```
You are an expert STEM/TEM microscopy analyst at the University of  
Tennessee Knoxville, Materials Science & Engineering department.  
When given a microscopy image or dataset: 1. Describe what you  
observe in the image 2. Identify crystal structure and orientation  
3. Call tools to make quantitative measurements 4. Search literature  
to contextualize findings 5. Report d-spacings and defect densities  
6. State confidence and limitations clearly 7. Cite all retrieved  
sources by DOI Never identify elements without tool verification.  
Think step by step before your final answer.
```

Chain-of-Thought & Few-Shot Prompting

Two techniques that dramatically improve reasoning accuracy on scientific tasks

Chain-of-Thought (CoT)

Forces the model to show its reasoning before concluding. Reduces hallucinations and improves accuracy on multi-step analysis.

```
# Add to system prompt:
"Think step by step. Write your
reasoning inside <thinking> tags,
then give your final analysis."

# Example agent output:
<thinking>
The bright columns suggest Ga sites
Spacing  $\approx 0.195$  nm  $\rightarrow$  GaN [0001]...
</thinking>

# Final answer, written after the <thinking> block:
Structure: GaN, wurtzite, [0001] zone axis
Defects: 3 dislocation cores detected
Confidence: high — matches 2 literature refs
```

Few-Shot Examples

Show the model 2–3 ideal examples of how to analyze a sample and call tools. Load them into message history before the user message.

```
# stem_analysis.json (few-shot)
[{"role": "user",
 "content": "Analyze SrTiO3_001.tif"},
 {"role": "assistant",
 "content": "<thinking>
Call detect_atomic_columns...
</thinking>
STO [001], d=0.390 nm"}]

# A second example teaches the edge case:
[{"role": "user",
 "content": "Analyze noisy_EELS.dat"},
 {"role": "assistant",
 "content": "Too noisy for confident ID.
Recommend recollection."}]
```

Structured Output & XML Tags

Two Anthropic-favored techniques that make a prompt's output (and input) unambiguous

Structured Output

Ask for a fixed JSON shape, then parse with `json.loads()`.

Turns free text into something your code can route and store.

Pairs naturally with the report format from the Pipeline slide.

```
# system.txt addition:
"Always respond in exactly this JSON shape:
  {"structure": "...", "confidence": 0-1,
   "citations": ["..."]}

# main.py
result = json.loads(response_text)
if result["confidence"] < 0.7:
    flag_for_review(result)
```

XML Tag Structuring

Wrap prompt sections in tags like `<protocol>` and `<observations>`.

Claude tracks XML-delimited sections without confusing them.

Essential once a prompt mixes static rules with live tool output.

```
# system.txt
"<protocol>
1. Describe the image
2. Identify crystal structure
3. Cite sources
</protocol>

<observations>
{tool_output}</observations>

Respond inside <analysis> tags."
```


Even More Prompting Techniques

Chain-of-thought and system prompts are a starting point, not the whole toolbox

Role & Persona Prompting

Calibrate vocabulary and confidence to the audience.
“Explain to a first-year grad student” reads very differently than “write for a peer-reviewed methods section.”

Self-Consistency

For an ambiguous structure ID, sample the same prompt 3–5 times and take the majority answer. Costs more tokens but catches one-off hallucinations.

Negative Examples

Show a wrong analysis next to the right one, and explain why it’s wrong. This calibrates overconfidence better than positive examples alone.

Prompt Chaining

Break “analyze this image” into separate prompts: describe → identify → verify → report. Each step is easier to get right, and easier to debug, than one giant prompt.

Prefilling the Response

Start the assistant’s turn for it — a leading { forces JSON, a leading “Structure:” forces a label — constraining format without even having to ask.

Don’t Bury Instructions

Put your most important constraint at both the very start and the very end of a long prompt. Models attend less to the middle of long contexts.

06

Fine-Tuning with LoRA

When, why, and how to adapt a model to electron microscopy

06

When to Fine-Tune

Exhaust these options first — fine-tuning is expensive and slow to iterate

Try These First

- X

Prompt engineering — better system prompt, CoT, role definition
- X

RAG — ground the model in your specific EM literature and protocols
- X

Few-shot examples — 3–5 annotated microscopy analyses in the prompt
- X

Structured output prompting — force JSON schemas before assuming the model can't format consistently

Good Reasons to Fine-Tune

- ✓

You have **500+ high-quality labeled examples** of EM image analysis
- ✓

You need a **specific output format** (structured crystallography reports, JCAMP-DX, etc.)
- ✓

Latency matters — a **fine-tuned 7B model** beats a prompted 70B model at inference cost
- ✓

Proprietary domain — **EM jargon, instrument brands, local materials** prompts can't fully capture

Rule of thumb: if prompting, RAG, and few-shot examples still leave a measurable gap after real testing — and you have the labeled data to prove it — that's when LoRA earns its compute budget. Until then, it's a distraction from shipping.

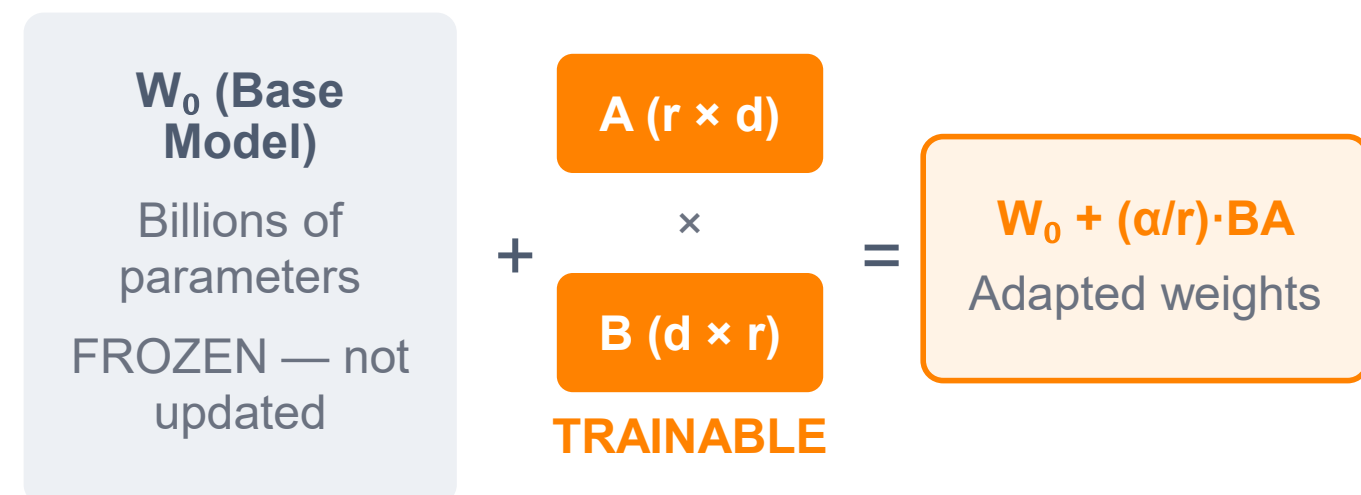
LoRA & QLoRA Explained

Train small adapter matrices — freeze the base model. Same quality at a fraction of the compute.

Why this works

Full fine-tuning updates every weight in the network — for a 7B-parameter vision model, that's 7 billion gradients to compute, store, and checkpoint every step.

LoRA assumes the weight update needed for a new domain is low-rank — it fits inside two small matrices instead of the full weight matrix. QLoRA adds 4-bit quantization of the frozen base, so the whole setup fits on a single GPU.



Rank **r = 16** typically updates only **~0.5% of parameters**. Set on `q_proj` and `v_proj` layers of the transformer.

```
from peft import LoraConfig, get_peft_model
from transformers import AutoModelForCausalLM, BitsAndBytesConfig

# QLoRA: 4-bit base + LoRA adapters
bnb = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",
)

model = AutoModelForCausalLM.from_pretrained(
    "Qwen/Qwen2.5-VL-7B-Instruct",
    quantization_config=bnb,
)

cfg = LoraConfig(
    r=16,                # rank
    lora_alpha=32,        # scaling factor
    target_modules=["q_proj", "v_proj"],
    lora_dropout=0.05,
)

model = get_peft_model(model, cfg)  # only ~0.5% of params updated!
```

Checkpointing: Save, Pause, Resume

The same word means two different things — a production agent needs both

Training Checkpoints

- Saved every N steps while fine-tuning (LoRA / QLoRA)
- Lets you keep the best-performing step, not just the last one
- Protects a multi-hour training run from crashes or loss spikes
- Evaluate each checkpoint before promoting it (see next slide)

```
trainer.save_model(f"checkpoint-{step}")  
  
# keep the checkpoint with the best eval score –  
# not necessarily the most recent one
```

Agent-Run Checkpoints

- Saved after every node or loop iteration during inference
- Crash mid-session? Resume from the last state, not from scratch
- Enables human-in-the-loop: pause, review, approve, continue
- Identified by a thread_id — one per microscopy session

```
app.invoke(inputs, config={  
    "configurable": {"thread_id": "grid3_run"}  
})
```

Rule of thumb: training checkpoints protect your compute investment; agent-run checkpoints protect a multi-hour TEM session's worth of instrument time and analysis progress. Checkpoint both.

Preparing Your Dataset

Quality beats quantity — 500 expert-annotated EM analyses outperform 10,000 noisy examples

```
# dataset.jsonl — one example per line
import json
example = {
  "messages": [
    {"role": "system",
     "content": "You are an expert STEM/TEM analyst..."},
    {"role": "user",
     "content": "Analyze this HAADF-STEM image of GaN"},
    {"role": "assistant",
     "content": "Wurtzite GaN [0001]. d=0.195 nm."}
  ]
}
with open("dataset.jsonl", "a") as f:
  f.write(json.dumps(example) + "\n")
```

Dataset Checklist

- **500–5,000** diverse, high-quality examples
- Cover all major **material families** (oxides, nitrides, metals, 2D materials)
- Include **negative examples** — poor quality images, ambiguous spectra
- Expert annotations with **DOI citations** in every answer
- Mix **HAADF + EELS + EDX + diffraction** modalities
- Hold out **15% for validation** — track loss and hallucination rate

Updating Your Model Over Time

Fine-tuning isn't a one-time event — it's a loop that runs alongside production

Log Every Run

Capture prompts, tool calls, and outputs from production agent sessions — today's runs are tomorrow's training signal.

Curate, Don't Dump

Filter to cases where the agent was wrong, uncertain, or corrected by a human. Quality beats raw volume for fine-tuning data.

Retrain on a Cadence

Re-run the LoRA fine-tuning job weekly or monthly as curated examples accumulate — not after every single session.

Version Everything

Tag each adapter (v1, v2, v3...) with its training data snapshot and eval scores. Never silently overwrite a working version.

Evaluate Before You Swap

Run the new version through the same eval suite from before — it must match or beat the current version to ship.

Canary, Then Rollback Plan

Roll the new version out to a small fraction of sessions first. Keep the previous version one config flag away.

07

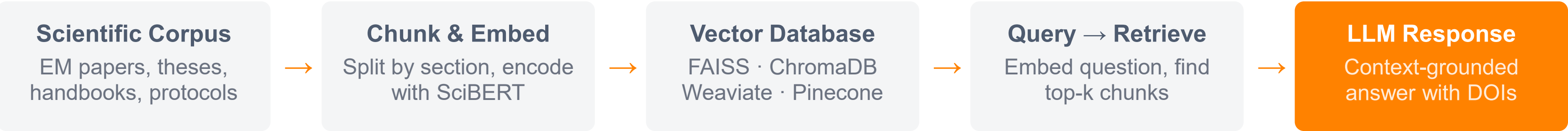
Building Your Microscopy Agent

RAG, full pipelines, and putting it all together

07

RAG for Scientific Literature

Ground your agent in verified EM knowledge — never rely on parametric memory alone



A Worked Retrieval

```
QUERY      "What accelerating voltage avoids knock-on damage in a beam-sensitive 2D sample?"
RETRIEVED  Egerton et al. 2010, Micron, Sec. 2.1 (cosine=0.84)
           "...operating below 80 kV avoids knock-on displacement damage in light-element 2D materials..."
ANSWER     Literature suggests operating below 80 kV for this sample class.
           [Egerton et al. 2010, DOI:10.1016/j.micron.2010.05.005]
```

Chunking Strategy

Chunk by section (Abstract, Methods, Results) — not arbitrary token counts. Preserve semantic context.

Hybrid Search

Combine dense (semantic) + sparse (BM25/keyword) retrieval. Pure semantic search misses exact EM terminology.

Citation Tracking

Always return source paper + DOI with every retrieval. The agent must cite evidence, not just state facts.

RAG in Code

Store your EM literature as embeddings once, then retrieve only the most relevant chunks at query time

Embed once, query many

Encode every paper chunk into a vector and store it. At question time you embed the query and fetch nearest neighbors.

Inject, don't fine-tune

Retrieved text is pasted into the prompt as grounding context. New papers just get added — no retraining needed.

```
# pip install chromadb sentence-transformers

import chromadb
from sentence_transformers import SentenceTransformer

model = SentenceTransformer(
    "all-MiniLM-L6-v2")

client = chromadb.Client()
coll = client.create_collection("em")

# index your paper chunks once

coll.add(documents=chunks,
         embeddings=model.encode(chunks),
         ids=ids)

def retrieve(query, n=3) -> str:
    q = model.encode([query])
    hits = coll.query(query_embeddings=q,
                     n_results=n)
    return "\n".join(hits["documents"][0])
```

The Agent Loop in Code

The while-loop that drives every STEM/TEM analysis session, step by step

Tool results -> “user” role

After execution, results are appended as a user message — the model gets another turn to reason.

stop_reason decides

“end_turn” means the agent wrote its final report; “tool_use” means it wants another tool.

```
def run_em_agent(user_message: str) -> str:
    system = load_prompt("system.txt")
    history = [{"role": "user", "content": user_message}]
    while True:
        response = client.messages.create(
            model=MODEL, system=system,
            tools=ALL_SCHEMAS, messages=history)
        history.append({"role": "assistant",
                        "content": response.content})
        if response.stop_reason == "end_turn":
            return response.content[-1].text
    results = []
    for block in response.content:
        if block.type != "tool_use": continue
        out = HANDLERS[block.name](**block.input)
        results.append({"type": "tool_result",
                        "tool_use_id": block.id, "content": out})
    history.append({"role": "user", "content": results})
```


main.py — Run Your Agent

The entry point ties it together; three terminal commands get it running on your own machine

The REPL loop

Read a line, pass it to `run_agent()`, print the reply, repeat. “quit” exits. This is your whole interface to start.

Three commands to launch

Install the SDK, drop your key into `.env`, run the script. A free key (with limits) comes from console.anthropic.com.

```
# main.py

from agent import run_agent

if __name__ == "__main__":
    print("Agent ready. Type quit to exit.")
    while True:
        msg = input("You: ").strip()
        if msg.lower() in {"quit", "exit"}:
            break
        print(f"Agent: {run_agent(msg)}")

# --- terminal -----
pip install anthropic python-dotenv
echo "ANTHROPIC_API_KEY=sk-ant-..." > .env
python main.py

# get a free key at console.anthropic.com
```

What Happens Each Iteration

One pass through the loop, broken into four steps

1

Send full history to the API

Every call includes the system prompt plus the entire message history. The model has no memory of its own — you carry it.

2

Model decides: done or tool?

`stop_reason = end_turn` means it wrote a final answer.
`stop_reason = tool_use` means it wants to call a tool first.

3

Your code runs the tool

Python executes the function with the model's inputs (e.g. `measure_lattice`) and captures the string result.

4

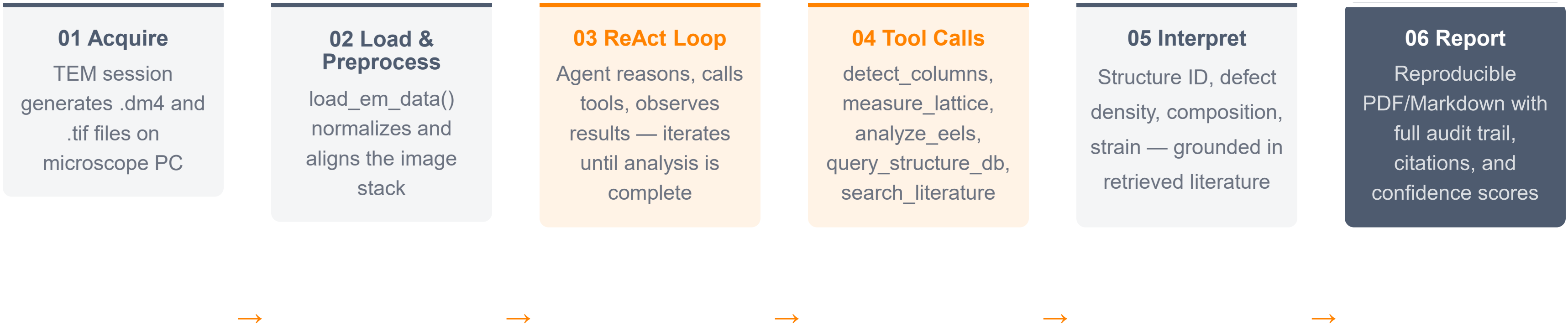
Append result, loop again

The result goes back as a user message. Call the API again — now the model knows what the tool returned.

Why this matters: because the model is stateless between calls, the loop is where all the intelligence accumulates. Lose the history and the agent forgets everything mid-task — which is exactly why you persist it (see Memory).

A Full STEM/TEM Agent Pipeline

End-to-end: from raw microscopy data to a reproducible, cited analysis report



What Step 06 Actually Outputs

```
=== STEM/TEM Analysis Report ===  
  
Sample:      GaN_2024-03-15_grid3.dm4  
Structure:   Wurtzite GaN, [0001] zone axis  
Lattice:     d = 0.195 nm (± 0.003) — matches reference value for GaN [0001]  
Defects:     3 threading dislocations, density 2.1×109 cm-2  
Composition: Ga 49.8% / N 50.2% (EDX, ±0.4%)  
Confidence:  High — cross-validated against 2 literature refs  
Citations:   Park 2019 (DOI:10.1016/j.actamat.2019.03.012); Chen 2021 (DOI:10.1021/acsnano.1c04522)  
Audit trail: 14 tool calls logged → runs/2024-03-15-grid3.json
```

Human checkpoints: after initial structure identification · after experiment re-design proposals · before any external data submission

Multi-Agent Patterns for Microscopy

Beyond a single agent: how specialized agents collaborate on EM workflows

Orchestrator + Workers

A planner agent decomposes the imaging goal and delegates to specialized workers — acquisition, spectral analysis, literature search.

Reflection Loop

An analysis agent proposes a structure ID; a critic agent checks it against diffraction data before the report is finalized — mirrors peer review.

Parallel Fan-Out

Run independent agents across multiple grid regions or samples simultaneously, then merge results — cuts wall-clock time for high-throughput screening.

Guardrail Agent

A fast, cheap model screens every tool call before it reaches an instrument — validates parameters and enforces safety limits.

Specialist Hand-off

A vision-language agent describes the image; a quantitative agent computes lattice parameters; a citation agent retrieves literature — one job each.

Human-in-the-Loop

Multi-agent systems pause for human approval before instrument changes, sample destruction, or external data submission.

Evaluation, Observability & Cost at Scale

You can't improve what you don't measure — and scale always has a price

Tracing

Log every LLM call — prompt, tool calls, latency, token cost — grouped into one trace per analysis session for full auditability.

Automated Evals

Build test cases from expert-labeled EM datasets; score every prompt change against ground-truth d-spacings and structure IDs.

A/B Prompt Testing

Run prompt variant A vs. B on the same microscopy dataset; compare accuracy and hallucination rate before rollout.

Model Tiering

Use a small model for routing and image triage; reserve large vision-language models for hard spectral interpretation — 10–50× cost difference.

Prompt Caching

Cache the long system prompt and instrument protocol context across calls in a session — cuts cost by up to 90%.

Batch Reprocessing

Use a batch API to re-analyze historical microscopy archives overnight at reduced cost — no real-time SLA required.

Key Takeaways

01 Agents are loops, not pipelines

Reason → Act → Observe. Design for iteration, self-correction, and goal-driven behavior.

02 smol-agents is a fast starting point

`pip install smolagents`. CodeAgent + Qwen2.5-VL runs locally on 4×H100 with no data leaving the lab.

03 Ground everything in RAG

Never trust parametric knowledge for crystallography or spectroscopy. Retrieve and cite sources always.

04 Prompt before you fine-tune

Try system prompts, few-shot, and RAG first. Fine-tune with LoRA/QLoRA only when you have 500+ quality examples.

05 Reproducibility is non-negotiable

Log model version, prompt, tool versions, and every intermediate result. Every agent run is an experiment.

06 Start small, iterate fast

One agent with 5 good microscopy tools beats 10 agents with 50 mediocre ones. Complexity is a liability.

Resources & Further Reading

Frameworks & Libraries

smolagents — huggingface.co/docs/smolagents

LangGraph — langchain-ai.github.io/langgraph

AutoGen — microsoft.github.io/autogen

peft (LoRA) — huggingface.co/docs/peft

hyperspy — hyperspy.org — EM data analysis in Python

Models for Microscopy

Qwen2.5-VL-7B/72B — vision-language, runs on 4×H100

Llama 3.1 70B — open weights, air-gapped deployments

Foundational Papers

ReAct: Yao et al., 2022 — Reasoning + Acting in LLMs

LoRA: Hu et al., 2021 — Low-Rank Adaptation

QLoRA: Dettmers et al., 2023 — 4-bit fine-tuning

Toolformer: Schick et al., 2023 — Self-supervised tool use

Science + AI

AI Scientist — Sakana AI, 2024

AtomAgent — autonomous EM analysis agent (2024)

M3GNet / CHGNET — universal force fields for structure sim

UNIVERSITY OF TENNESSEE KNOXVILLE

Questions?

Sheryl L. Sanchez, PhD

Materials Science & Engineering · UTK